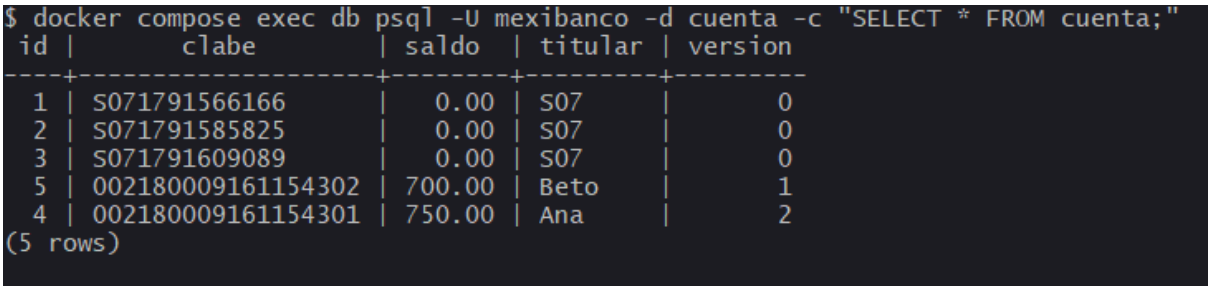
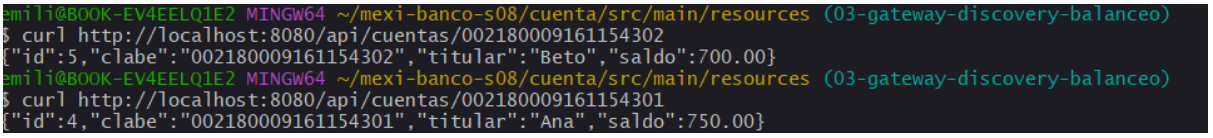
Bitácora Labs08: Que la transferencia no deje dinero en el aire

Parte 0 · Ver el problema antes de arreglarlo

Para comprobar el problema de las transacciones distribuidas sin compensación, primero revisé los saldos iniciales directamente en la base de datos de PostgreSQL ejecutando un query en el contenedor db. Tomé como caso de prueba a Ana (CLABE: `002180009161154301`) con un saldo inicial de $750.00, y a Beto (CLABE: `002180009161154302`) con $700.00.



```
$ docker compose exec db psql -U mexibanco -d cuenta -c "SELECT * FROM cuenta;"
 id |       clabe        | saldo  | titular | version
----+--------------------+--------+---------+---------
  1 | S071791566166      |   0.00 | S07     |       0
  2 | S071791585825      |   0.00 | S07     |       0
  3 | S071791609089      |   0.00 | S07     |       0
  5 | 002180009161154302 | 700.00 | Beto    |       1
  4 | 002180009161154301 | 750.00 | Ana     |       2
(5 rows)
```



```
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s08/cuenta/src/main/resources (03-gateway-discovery-balanceo)
$ curl http://localhost:8080/api/cuentas/002180009161154302
{"id":5,"clabe":"002180009161154302","titular":"Beto","saldo":700.00}
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s08/cuenta/src/main/resources (03-gateway-discovery-balanceo)
$ curl http://localhost:8080/api/cuentas/002180009161154301
{"id":4,"clabe":"002180009161154301","titular":"Ana","saldo":750.00}
```

Inmediatamente después, lancé una transferencia de $50.00 de la cuenta de Ana a la cuenta de Beto a través del gateway. Mientras la petición estaba en espera (durante los 20 segundos de pausa), simulé la caída del sistema apagando el contenedor de las cuentas desde otra terminal con `docker compose stop cuenta`.

Una vez que la petición falló, volví a levantar el servicio (`docker compose start cuenta`) y volví a consultar los saldos en la base de datos: el cargo de $50.00 sí se le aplicó a Ana (quedó en $700.00), pero el abono nunca le llegó a Beto (se quedó en $700.00).

```
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s08/cuenta/src/main/resources (03-gateway-discovery-balanceo)
$ PAUSA_ENTRE_CARGO_Y_ABONO_MS=20000 docker compose up -d transferencia
[+] up 3/3
 ✔ Container mexi-banco-v06-db-1            Healthy
 ✔ Container mexi-banco-v06-discovery-1     Healthy
 ✔ Container mexi-banco-v06-transferencia-1 Running
```

```
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s08 (03-gateway-discovery-balanceo)
$ docker compose stop cuenta
[+] stop 3/3
 ✔ Container mexi-banco-v06-cuenta-3 Stopped                                3.3s
 ✔ Container mexi-banco-v06-cuenta-2 Stopped                                3.3s
 ✔ Container mexi-banco-v06-cuenta-1 Stopped                                3.3s
```

POST http://localhost:8080/api/transferencias

Docs Params Authorization Headers 10 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQ

```
1 {
2   "claveOrigen": "002180009161154301",
3   "claveDestino": "002180009161154302",
4   "monto": 50.00
5 }
```

Body Cookies Headers 2 Test Results 0/1

{ } JSON Preview Debug with AI

```
1 {
2   "timestamp": "2026-10-16T20:31:59.561+00:00",
3   "path": "/api/transferencias",
4   "status": 504,
5   "error": "Gateway Timeout",
6   "requestId": "4c5527bb-925"
7 }
```

```
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s08/cuenta/src/main/resources (03-gateway-discovery-balanceo)
$ docker compose exec db psql -U mexibanco -d cuenta -c "SELECT * FROM cuenta;"
 id |       clabe        | saldo  | titular | version
----+--------------------+--------+---------+---------
  1 | S071791566166      |   0.00 | S07     |       0
  2 | S071791585825      |   0.00 | S07     |       0
  3 | S071791609089      |   0.00 | S07     |       0
  5 | 002180009161154302 | 700.00 | Beto    |       1
  4 | 002180009161154301 | 700.00 | Ana     |       3
(5 rows)
```

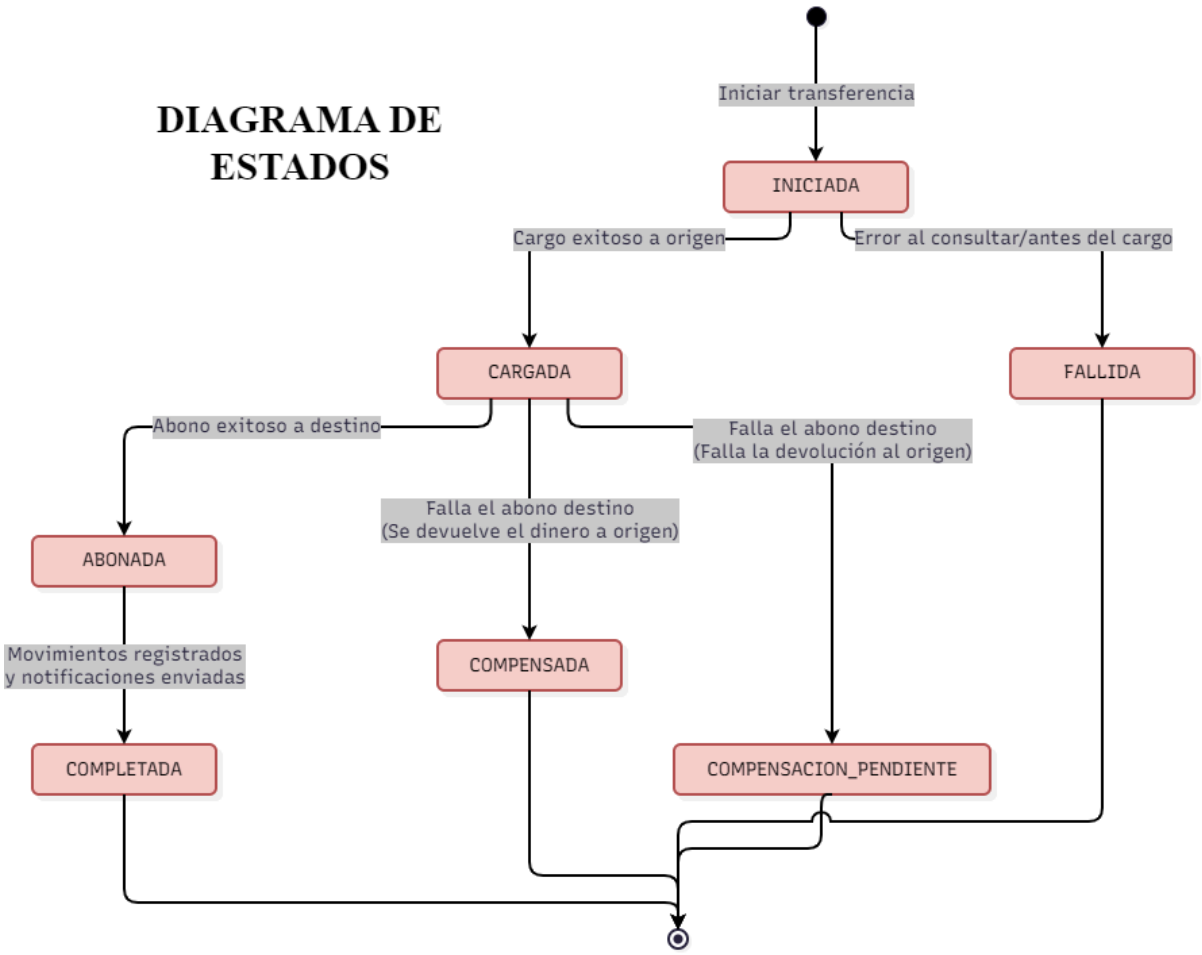
¿Por qué pasó esto? El dinero se quedó "en el aire" porque el código original de la transferencia confirma cada salto HTTP por separado; al caerse el servicio de cuentas, el abono falló, pero no existe ningún mecanismo que deshaga el cargo que ya se le había hecho exitosamente a la cuenta de origen.

2. Tu saga: los pasos y la compensación

Para resolver el problema de las transacciones distribuidas, implementé una **Saga Orquestada** donde el TransferenciaService funciona como el orquestador central. Definí los

siguientes estados para rastrear exactamente en qué paso se encuentra la transferencia en la base de datos:

- **INICIADA**: Se registró la intención de transferencia en la base de datos antes de mover dinero.
- **CARGADA**: Se completó exitosamente el cargo a la cuenta de origen mediante la llamada HTTP.
- **ABONADA**: Se completó exitosamente el abono a la cuenta destino.
- **COMPLETADA**: Se registraron los movimientos finales y notificaciones.
- **COMPENSADA**: El abono al destino falló, por lo que la saga ejecutó una transacción de compensación (un abono) para devolver el dinero a la cuenta origen.
- **COMPENSACION_PENDIENTE**: Ocurrió un error crítico donde falló tanto el abono destino como el intento de devolver el dinero al origen.
- **FALLIDA**: La transacción falló antes de realizar el primer cargo.



3. El código clave

Para lograr esta máquina de estados, realicé modificaciones en dos archivos principales:

1. Entidad Transferencia: Agregué el atributo para persistir el estado en Transferencia.java

```
31      private String estado;
32
33      public String getEstado() { return estado; }
34      public void setEstado(String estado) { this.estado = estado; }
35
```

2. El Orquestador (TransferenciaService): Envolví la lógica de las peticiones HTTP en un bloque try-catch. Si ocurre un error después de que el estado cambió a CARGADA, el catch actúa como el mecanismo de compensación devolviendo el dinero al origen. También inyecté una variable de entorno para simular fallas controladas.

```
try {
    // 2. Hacer el CARGO
    CuentaRemota origen = cuentas.cargar(claveOrigen, monto);
    transferencia.setEstado("CARGADA");
    transferencias.save(transferencia);

    pausaDeDemo();

    // 3. Forzar error
    if (claveDestino.equals(clabeFallaAbono)) {
        throw new RuntimeException("Falla simulada: Abono rechazado por variable de entorno");
    }

    // 4. Hacer el ABONO
    CuentaRemota destino = cuentas.abonar(claveDestino, monto);
    transferencia.setEstado("ABONADA");
    transferencias.save(transferencia);

    // 5. Registrar movimientos finales si todo salió bien
    MovimientoRemoto cargo = movimientos.registrar(origen.clabe(), TipoMovimiento.TRANSFERENCIA_ENVIADA,
            monto.negate(), origen.saldo(), "a " + destino.clabe());
    MovimientoRemoto abono = movimientos.registrar(destino.clabe(), TipoMovimiento.TRANSFERENCIA_RECIBIDA,
            monto, destino.saldo(), "de " + origen.clabe());

    transferencia.setEstado("COMPLETADA");
    transferencias.save(transferencia);

    notificaciones.notificar(cargo);
    notificaciones.notificar(abono);

    return transferencia;

} catch (Exception e) {
```

```
} catch (Exception e) {
    // --- INICIA SAGA DE COMPENSACIÓN ---
    if ("CARGADA".equals(transferencia.getEstado())) {
        try {
            CuentaRemota origenDevolucion = cuentas.abonar(claveOrigen, monto);
            movimientos.registrar(claveOrigen, TipoMovimiento.TRANSFERENCIA_RECIBIDA,
                    monto, origenDevolucion.saldo(), "DEVOLUCIÓN por falla hacia " + claveDestino);

            transferencia.setEstado("COMPENSADA");
            transferencias.save(transferencia);

            throw new ReglaDeNegocioException("Falló el abono destino. Se compensó regresando el dinero a origen. Error original: " + e.getMessage());

        } catch (Exception exCompensacion) {
            transferencia.setEstado("COMPENSACION_PENDIENTE");
            transferencias.save(transferencia);
            throw new RuntimeException("ERROR CRÍTICO: Falló abono y falló compensación. Dinero en el aire.", exCompensacion);
        }
    }

    // Si falló en "INICIADA" (antes del cargo)
    transferencia.setEstado("FALLIDA");
    transferencias.save(transferencia);
    throw new ReglaDeNegocioException("La transferencia falló antes de mover dinero: " + e.getMessage());
}
```

4. La compensación funcionando

Para probar que la saga recupera el dinero correctamente sin tener que apagar los contenedores a la fuerza, utilicé la variable de entorno MEXIBANCO_DEMO_CLABE_FALLA_ABONO=002180009161154302 (la CLABE de Beto) en el docker-compose.yml.

Lancé una transferencia de $50.00 desde la cuenta de Ana hacia la cuenta de Beto. Como se observa en la evidencia, el gateway respondió con un código HTTP 422 indicando claramente que el abono falló y se ejecutó la compensación.

Al consultar la base de datos cuenta, se puede verificar que el dinero de Ana no se perdió en el aire. Su saldo inicial era de $700.00 (versión 5) y terminó exactamente en $700.00 (versión 7). El salto de dos versiones demuestra que el dinero salió de su cuenta (versión 6) y la saga se lo devolvió automáticamente (versión 7).

```
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s08 (03-gateway-discovery-balanceo)
$ docker compose exec db psql -U mexibanco -d cuenta -c "SELECT * FROM cuenta;"
 id |       clabe        | saldo  | titular | version
----+--------------------+--------+---------+---------
  1 | S071791566166      |   0.00 | S07     |       0
  2 | S071791585825      |   0.00 | S07     |       0
  3 | S071791609089      |   0.00 | S07     |       0
  5 | 002180009161154302 | 700.00 | Beto    |       1
  4 | 002180009161154301 | 700.00 | Ana     |       5
(5 rows)

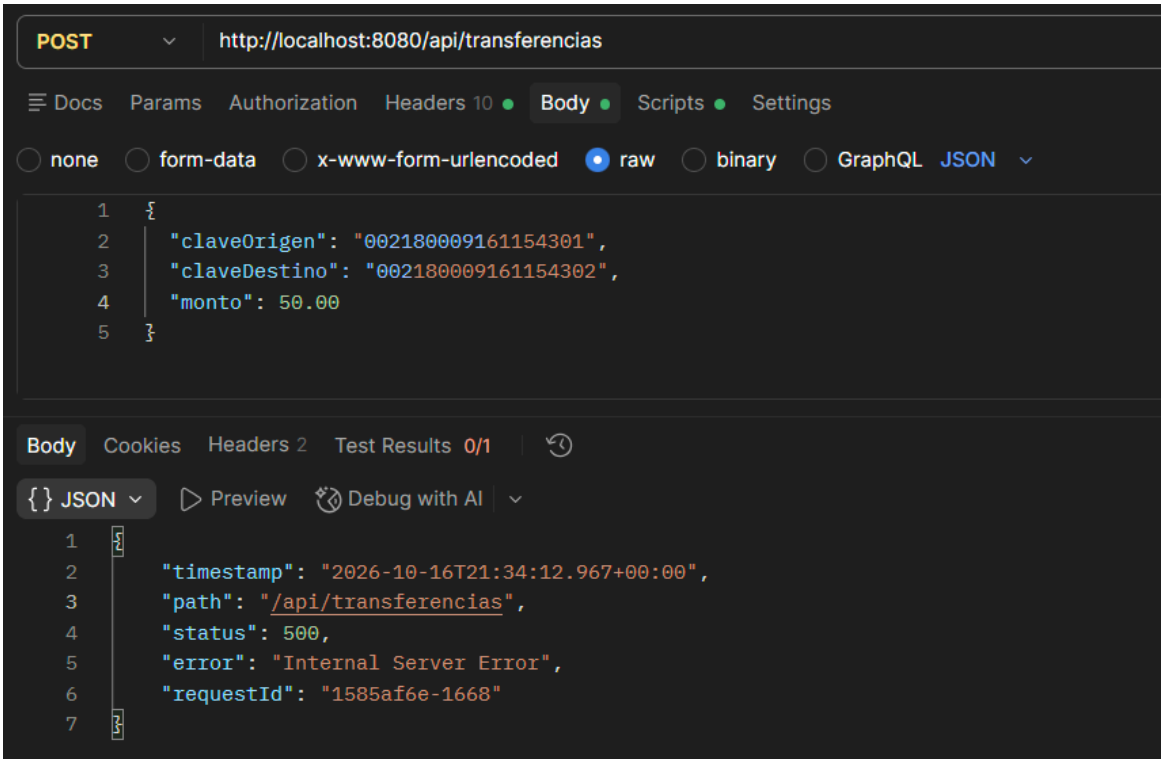

emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s08 (03-gateway-discovery-balanceo)
$ curl -X POST http://localhost:8080/api/transferencias      -H "Content-Type: application/json"
  -d '{
          "claveOrigen": "002180009161154301",
          "claveDestino": "002180009161154302",
          "monto": 50.00
        }'
{"type":"about:blank","title":"Unprocessable Entity","status":422,"detail":"Falló el abono destino.
Se compensó regresando el dinero a origen. Error original: Falla simulada: Abono rechazado por varia
ble de entorno","instance":"/transferencias"}
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s08 (03-gateway-discovery-balanceo)
$ docker compose exec db psql -U mexibanco -d cuenta -c "SELECT * FROM cuenta;"
 id |       clabe        | saldo  | titular | version
----+--------------------+--------+---------+---------
  1 | S071791566166      |   0.00 | S07     |       0
  2 | S071791585825      |   0.00 | S07     |       0
  3 | S071791609089      |   0.00 | S07     |       0
  5 | 002180009161154302 | 700.00 | Beto    |       1
  4 | 002180009161154301 | 700.00 | Ana     |       7
(5 rows)
```

5. Cuando la compensación también falla

Para probar el peor escenario posible (que el sistema falle justo cuando está intentando devolver el dinero), lancé una transferencia hacia la CLABE destino configurada para fallar. Sin embargo, durante la pausa de 20 segundos inyectada entre el cargo y el abono, apagué manualmente los contenedores del servicio cuenta.

Esto provocó que el bloque catch de la saga, al intentar ejecutar la compensación (la devolución al origen), también fallara porque el servicio ya no estaba disponible. Como se observa en la evidencia de Postman, el gateway devolvió un error 500 Internal Server Error.

Al revisar la base de datos, se comprueba que el dinero quedó momentáneamente "en el aire". El saldo de Ana bajó a $700.00 (se le aplicó el cargo) en su versión 7, pero la compensación no pudo concretarse. El bloque de error crítico de la saga atrapó esta situación y marcó la transferencia con el estado COMPENSACION_PENDIENTE.




```
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s08 (03-gateway-discovery-balanceo)
$ PAUSA_ENTRE_CARGO_Y_ABONO_MS=20000 docker compose up -d transferencia
[+] up 3/3
 ✔ Container mexi-banco-v06-discovery-1     Healthy                    6.3s
 ✔ Container mexi-banco-v06-db-1            Healthy                    6.3s
 ✔ Container mexi-banco-v06-transferencia-1 Started                    8.6s

emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s08 (03-gateway-discovery-balanceo)
$ docker compose exec db psql -U mexibanco -d cuenta -c "SELECT * FROM cuenta;"
 id |       clabe        | saldo  | titular | version
----+--------------------+--------+---------+---------
  1 | S071791566166      |   0.00 | S07     |       0
  2 | S071791585825      |   0.00 | S07     |       0
  3 | S071791609089      |   0.00 | S07     |       0
  5 | 002180009161154302 | 700.00 | Beto    |       1
  4 | 002180009161154301 | 700.00 | Ana     |       7
(5 rows)
```

¿Cómo terminaría esta transferencia atascada? Para resolver las transferencias en estado COMPENSACION_PENDIENTE, implementaría un proceso asíncrono en segundo plano (como un *cron job* o una tarea @Scheduled en Spring Boot). Este proceso consultaría periódicamente la base de datos buscando transferencias en este estado crítico y volvería a intentar la llamada HTTP de abono hacia la cuenta origen. Si el reintento es exitoso, actualizaría el estado finalmente a COMPENSADA.

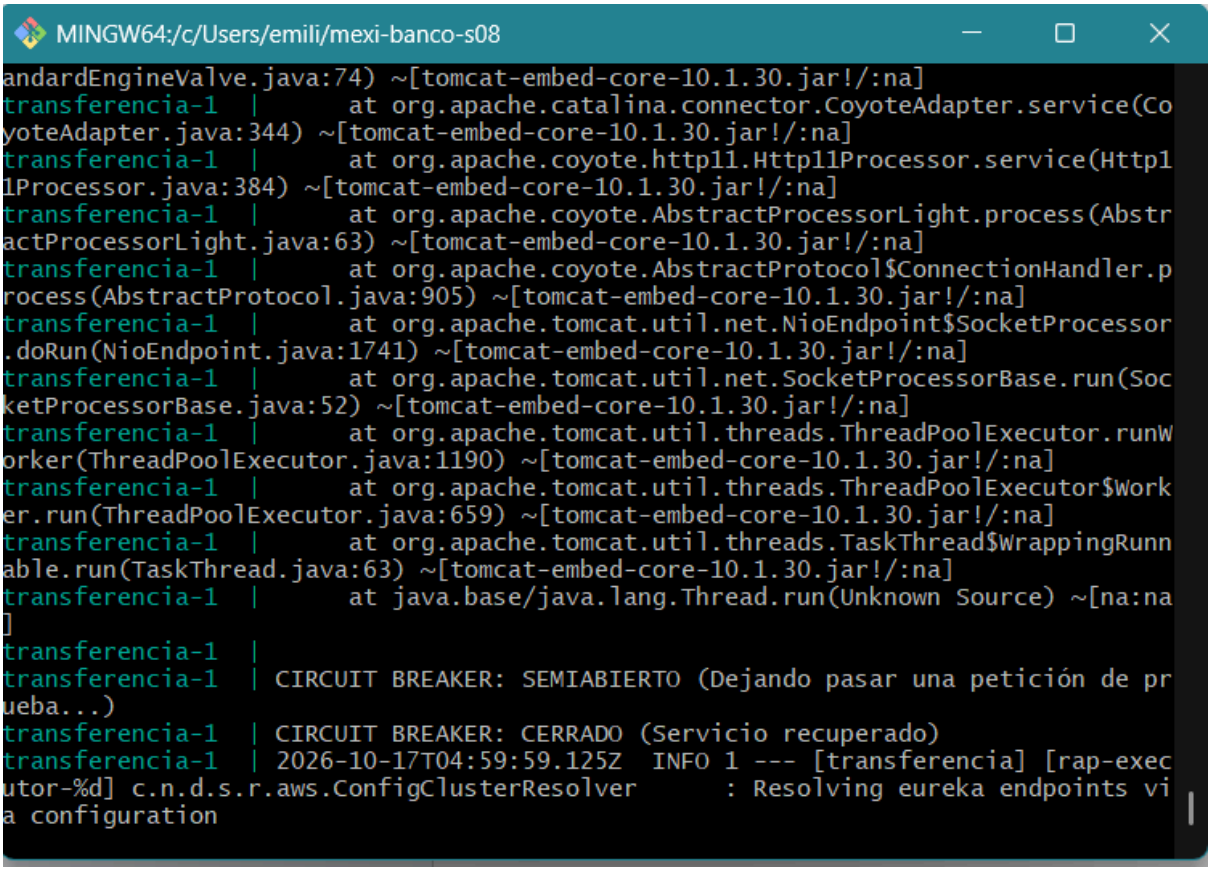
¿Por qué la compensación debe ser idempotente? Es fundamental que la operación de compensación sea idempotente porque las fallas de red son impredecibles. Podría ocurrir que el abono de compensación sí llegue a registrarse en el servicio de cuenta, pero la respuesta de confirmación se pierda en la red antes de llegar al orquestador. Si el proceso de reintento en segundo plano vuelve a lanzar la compensación a ciegas, el sistema le devolvería el dinero al cliente dos o más veces. Al diseñar el abono como una operación idempotente (por ejemplo, enviando el ID único de la transferencia), el servicio de cuenta sabría que esa compensación específica ya se ejecutó y no sumaría el saldo nuevamente.

6. El circuit breaker en las llamadas a cuenta

Para proteger al orquestador de quedarse bloqueado esperando respuestas si el servicio de cuentas se cae, decidí implementar un **Circuit Breaker a mano** (creando una clase personalizada CuentaCircuitBreaker con lógica de estados, contador de fallas y temporizador) para obtener los puntos extra de la práctica.

El ciclo de vida y los tres estados se comprobaron de la siguiente manera:

- **CERRADO:** Es el estado inicial y normal. Las peticiones hacia la cuenta fluyen sin interrupciones.
- **ABIERTO:** Apagué el contenedor cuenta y mandé varias transferencias seguidas. Mi clase registró las fallas y, al llegar al límite de 3 errores, abrió el circuito. A partir de ahí, las siguientes peticiones hacían un "fallo rápido" (fast-fail), devolviendo un error inmediato de que el servicio estaba caído sin intentar hacer la llamada HTTP, ahorrando recursos y evitando timeouts.
- **SEMIABIERTO:** Volví a encender el contenedor de cuenta. Esperé el tiempo de enfriamiento configurado (10 segundos) y lancé una transferencia más. Como se observa en la evidencia de los logs de la terminal, el circuito dejó pasar esta petición de prueba cambiando a estado **SEMIABIERTO**. Al ver que la llamada a la base de datos fue exitosa, imprimió el restablecimiento del servicio y volvió al estado **CERRADO**.



7. Lo que la saga no te da (Falta de Aislamiento)

A diferencia del @Transactional de un monolito que cumple con las propiedades ACID, la Saga Orquestada sacrifica el Aislamiento (Isolation). Si alguien consulta el saldo de la cuenta origen exactamente en el periodo de tiempo que transcurre entre que se hace el cargo inicial y ocurre la compensación, verá que el dinero ya no está en la cuenta origen, aunque la transferencia aún no se haya completado ni haya llegado al destino. Si la saga falla y compensa devolviendo el dinero minutos después, el saldo de la cuenta origen "reaparecerá" mágicamente, lo que podría confundir al usuario o a otros servicios que hayan leído ese saldo intermedio inconsistente.